

Model Context Protocol

Interview-Ready Revision Sheet

Source	CampusX YouTube — MCP Trilogy Playlist
Topics	Why MCP · Architecture · Lifecycle · Local Server · Remote Server · MCP Client
Coverage	Definitions · Architecture · Code Examples · Interview Q&A
Format	Interview-Ready Revision PDF

Table of Contents

1. What is MCP? — Definition & Origin
2. Why MCP? — The Problem of Fragmentation
3. MCP vs Function Calling vs REST API
4. MCP Architecture — Hosts, Clients, Servers
5. MCP Primitives — Tools, Resources, Prompts
6. MCP Lifecycle — Connection to Termination
7. Transport Layer — STDIO vs Streamable HTTP (SSE)
8. Building Local MCP Servers (FastMCP)
9. Building Remote MCP Servers
10. Building MCP Clients
11. MCP Inspector — Debugging Tool
12. MCP Ecosystem & Real-world Use Cases
13. Interview Q&A; — 30 Questions
14. Quick-Reference Cheat Sheet

1. What is MCP? — Definition & Origin

Model Context Protocol (MCP) is an **open, standardised protocol** introduced by Anthropic (November 2024) that defines a universal, vendor-neutral way for AI applications to connect to external tools, data sources and services. It is now hosted by the **Linux Foundation** and open to community contributions.

Often called the **"USB-C for AI"** — just as USB-C standardised how devices connect to peripherals, MCP standardises how LLMs connect to the world outside their weights.

Key Properties

- Language & vendor agnostic — works with any LLM (Claude, GPT, Gemini, ...).
- Client-Server architecture built on **JSON-RPC 2.0**.
- Three primitives: **Tools** (actions), **Resources** (data), **Prompts** (templates).
- Two transports: **STDIO** (local) and **Streamable HTTP / SSE** (remote).
- Enables full agentic workflows: reason → retrieve → act → iterate.

■ *MCP is NOT a replacement for the LLM itself; it extends what the LLM can DO and SEE.*

2. Why MCP? — The Problem of Fragmentation

The Evolution Story (CampusX — 'The Why')

- **Wave 1 — Standalone LLMs:** Chat UIs, summarisation. LLMs are isolated; no tool access.
- **Wave 2 — Function Calling:** LLMs can invoke pre-defined functions. Works but requires custom glue code per integration.
- **Wave 3 — Tool Proliferation:** Many tools, but each LLM-app pair needs its own M×N adapter layer.
- **Wave 4 — MCP:** A single, standardised protocol. Any client talks to any server. M+N instead of M×N.

The Copy-Paste Hell Problem

Before MCP, a developer using AI in their workflow had to manually copy context (code, documents, database records) into the chat window. This is error-prone, breaks continuity, and does not scale to autonomous agents. MCP solves this by making the AI *pull* the right context itself from authoritative servers.

Problem with Naked Tool Calling (Pre-MCP)

- Every AI application had to implement its own tool-calling schema.
- Tools were tightly coupled to one specific LLM or framework.
- No discovery mechanism — the app had to hard-code every tool.
- Security & auth were ad-hoc and inconsistent.
- Sharing tools across teams / products was nearly impossible.

■ ■ Without MCP, adding a new data source (e.g., Notion) to 5 different AI apps means writing 5 separate integrations. With MCP, you write ONE MCP server and all 5 apps connect automatically.

3. MCP vs Function Calling vs REST API

Dimension	Function Calling	REST API	MCP
Standardisation	LLM-specific schema	HTTP standard	Open, vendor-neutral protocol
Discovery	Hard-coded in prompt	Manual / OpenAPI	Dynamic via tools/list endpoint
Who controls call	Model (decides when)	Developer (explicit call)	Model (tools) or App (resources)
Transport	Embedded in LLM API	HTTP/S	STDIO or Streamable HTTP
Context sharing	Manual	Manual	Resources & Prompts built-in
Reusability	Low — per-app logic	Medium	High — any MCP client can use
Auth	Ad-hoc	OAuth / API key	Server-side, standardised
Statefulness	Stateless per call	Stateless	Stateful session lifecycle

■ Interview angle: MCP does NOT replace function calling — it STANDARDISES and EXTENDS it into a full protocol with lifecycle, auth, and discovery.

4. MCP Architecture — Hosts, Clients, Servers

Core Components

Component	Definition	Examples
MCP Host	The AI application that wants to use external tools	ChatGPT, Claude, OpenAI GPT, Gemini, Copilot, custom AI
MCP Client	Protocol client inside the Host. Maintains a 1:1 stable connection to an MCP Server	Bedrock, Claude Desktop, MCP Browser extension
MCP Server	Lightweight process that exposes capabilities (tools, resources, prompts)	Local file explorer, GitHub MCP, Jupyter MCP, etc.
Local Data Sources	Files, databases, services on the user's machine	SQLite DB, local files, Docker containers
Remote Services	Cloud APIs and external platforms the MCP Server connects to	GitHub API, Slack API, Google Drive

Data Flow Diagram (Text)

User → Host (AI App) → MCP Client → MCP Server → Data Source / API

The Host does NOT talk to data sources directly. It talks to the MCP Client, which forwards requests to the appropriate MCP Server, which then accesses the underlying data.

Two Protocol Layers

- **Data Layer:** Defines the JSON-RPC 2.0 message format, lifecycle management, and the three primitives (tools, resources, prompts).
- **Transport Layer:** Defines the physical channel — STDIO for local, Streamable HTTP for remote. Described in Section 7.

5. MCP Primitives — Tools, Resources, Prompts

Overview Table

Primitive	Definition	Who Controls	Read/Write	Endpoint
Tools	Executable functions the LLM can invoke to perform actions.	Model controlled (LLM decides when to use)	Read/Write	tools/list tools/call
Resources	Read-only data sources that provide context to the LLM.	Application controlled (Host decides when to use)	Read-only	resources/list resources/read
Prompts	Reusable, pre-crafted instruction templates that guide the LLM.	User controlled (User selects template)	Read-only	prompts/list prompts/get

Tools — Deep Dive

Tools are the **action verbs** of MCP. The LLM autonomously decides when to call a tool based on the conversation. Each tool is defined with a name, description, and a **JSON Schema** for its inputs.

```
# FastMCP — Defining a Tool (Python)
from fastmcp import FastMCP
import sqlite3

mcp = FastMCP('expense-tracker')

@mcp.tool()
def add_expense(amount: float, category: str, description: str) -> str:
    """Add a new expense entry to the database."""
    conn = sqlite3.connect('expenses.db')
    conn.execute(
        'INSERT INTO expenses (amount, category, description) VALUES (?, ?, ?)'
        (amount, category, description)
    )
    conn.commit()
    conn.close()
    return f'Expense of {amount} added under {category}.'
```

Resources — Deep Dive

Resources expose **read-only context**. The Host application makes them available; the LLM can reference them but cannot modify them through the resource interface. Think of them as a librarian handing over reference books.

```
# FastMCP — Defining a Resource
@mcp.resource('config://categories')
def get_categories() -> str:
    """Return allowed expense categories as JSON."""
    import json
    categories = ['Food', 'Travel', 'Utilities', 'Entertainment', 'Health']
    return json.dumps(categories)
```

Prompts — Deep Dive

Prompts are **server-defined instruction templates**. A user picks a prompt from the Host UI (like a slash-command) and the server fills in the template with the right instructions and context for the LLM.

```
# FastMCP — Defining a Prompt
from mcp.types import PromptMessage, TextContent
@mcp.prompt()
def monthly_summary_prompt(month: str) -> list[PromptMessage]:
    """Generate a monthly expense analysis prompt """
    return [
        PromptMessage(
            role='user',
            content=TextContent(
                type='text',
                text=f'Analyse all expenses for {month} '
                    f'Group by category, show totals, and flag anomalies '
            )
        )
    ]
```

6. MCP Lifecycle — Connection to Termination

Every MCP session follows a **five-phase lifecycle**. Understanding this is critical for debugging and interviews.

Phase	Name	What Happens
1	Discovery	Host finds available MCP servers (config file, registry, env vars). For local: path to executable. For remote: endpoint URL.
2	Initialisation	Client sends initialize request with its protocol version and capabilities. Server responds with its version and supported features.
3	Capability Exchange	Client calls tools/list, resources/list, prompts/list to discover what the server offers. Both sides now know what the other can do.
4	Active Session	Normal operation — LLM makes tool calls, reads resources, selects prompts. Server sends notifications for tool results, resource updates, etc.
5	Termination	Client sends shutdown signal. Server cleans up (closes DB connections, etc.). Process exits. For remote: connection is closed.

■ The initialize handshake uses protocol versioning — if client and server versions are incompatible, the session is rejected before any data flows.

7. Transport Layer — STDIO vs Streamable HTTP (SSE)

Transport Overview

Aspect	STDIO (Standard I/O)	Streamable HTTP / SSE
Use case	Local server, same machine	Remote server, multi-client, cloud
How it works	Client spawns server as child process; JSON-RPC over stdout/stderr.	Client connects via HTTP; server streams responses via SSE.
Latency	Extremely low (in-process)	Network latency
Multi-client	No — 1:1 relationship	Yes — multiple clients per server instance
Auth	OS-level process isolation	OAuth 2.0 / Bearer tokens / API keys
Deployment	Desktop apps (Claude Desktop)	Cloud, Docker, SaaS platforms
Logging	Must use stderr (stdout = protocol stream)	Can use any logging framework
Example clients	Claude Desktop, local agents	Claude.ai, Cursor remote, web agents

One-Sentence Decision Rule

"If the user of the AI client also controls the machine the server runs on → use STDIO. If they don't → use Streamable HTTP."

■■ SSE (Server-Sent Events) was MCP's original remote transport (spec 2024-11-05) requiring two endpoints. It was replaced by Streamable HTTP in spec 2025-03-26. Many tutorials still show SSE — know both for interviews.

Running a FastMCP server in STDIO mode (default)


```
# server.py
from fastmcp import FastMCP
mcp = FastMCP('mv-server')
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b
if __name__ == '__main__':
    mcp.run() # defaults to STDIO
```

```
# Running a FastMCP server in Streamable HTTP mode (remote)
if __name__ == '__main__':
    mcp.run(transport='streamable-http' host='0.0.0.0' port=8000)
# Clients connect to http://your-host:8000/mcp
```

8. Building Local MCP Servers (FastMCP)

FastMCP vs Official MCP SDK

	Official MCP SDK (mcp)	FastMCP
Abstraction	Low-level, verbose boilerplate	High-level decorators, minimal code
Learning curve	Steep	Gentle — Pythonic
Production	Full control	Built on top of official SDK
FastAPI compat	No built-in	Can convert FastAPI routes to MCP tools
Install	<code>pip install mcp</code>	<code>pip install fastmcp</code>

■ *FastMCP IS the official high-level SDK now maintained by Anthropic. It is NOT a third-party wrapper.*

Full Local Server Example — Expense Tracker (CampusX Project)

```
# expense_tracker_server.py
from fastmcp import FastMCP
import sqlite3
from datetime import date

mcp = FastMCP('expense-tracker')
DB = 'expenses.db'

def init_db():
    conn = sqlite3.connect(DB)
    conn.execute("""CREATE TABLE IF NOT EXISTS expenses
                    (id INTEGER PRIMARY KEY AUTOINCREMENT
                     amount REAL category TEXT
                     description TEXT date TEXT)""")
    conn.commit()
    conn.close()

@mcp.tool()
def add_expense(amount: float, category: str,
                description: str, expense_date: str = '') -> str:
    """Add a new expense. date format: YYYY-MM-DD"""
    if not expense_date:
        expense_date = str(date.today())
    conn = sqlite3.connect(DB)
    conn.execute('INSERT INTO expenses VALUES (NULL, ?, ?, ?, ?)'
                  (amount, category, description, expense_date))
    conn.commit()
    conn.close()
    return f'Added: {category} - {amount} on {expense_date}'

@mcp.tool()
def list_expenses(start_date: str = '' and date: str = '') -> str:
    """List expenses, optionally filtered by date range"""
    conn = sqlite3.connect(DB)
    if start_date and end_date:
        rows = conn.execute(
            'SELECT * FROM expenses WHERE date BETWEEN ? AND ?'
            (start_date, end_date))
        fetchall()
    else:
```

```

        rows = conn.execute('SELECT * FROM expenses').fetchall()
    conn.close()

    return json.dumps([{'id':r[0], 'amount':r[1], 'category':r[2],
                        'desc':r[3], 'date':r[4]} for r in rows])

@mcp.tool()
def summarise_expenses() -> str:
    """Return total spend per category"""
    conn = sqlite3.connect(DB)
    rows = conn.execute(
        'SELECT category, SUM(amount) FROM expenses GROUP BY category'
    ).fetchall()
    conn.close()
    return json.dumps([r[0], r[1]] for r in rows)

@mcp.resource('config://categories')
def categories() -> str:
    """Allowed expense categories"""
    return json.dumps(['Food', 'Travel', 'Utilities', 'Health', 'Entertainment'])

if __name__ == '__main__':
    init_db()
    mcp.run() # STDIO - for Claude Desktop

```

Connecting to Claude Desktop (claude_desktop_config.json)

```

# ~/config/Claude/claude_desktop_config.json
{
  "mcpServers": {
    "expense-tracker": {
      "command": "python"
      "args": ["/absolute/path/to/expense_tracker_server.py"]
      "env": {}
    }
  }
}

```

9. Building Remote MCP Servers

A **remote MCP server** runs on a separate machine (cloud VM, container, serverless function) and is accessible over the network via Streamable HTTP. Multiple MCP clients can connect to a single remote server simultaneously.

Converting the Local Server to Remote

```
# remote expense server.py (only change from local)
if __name__ == '__main__':
    init_dh()
    mcp.run(
        transport='streamable-http',
        host='0.0.0.0',
        port=8000
    )
# Endpoint: http://<your-ip>:8000/mcp
```

Connecting Claude Desktop to a Remote Server

```
# claude_desktop_config.json - remote
{
  "mcpServers": {
    "remote-expense-tracker": {
      "command": "npx"
      "args": ["-y", "mcp-remote", "http://your-server-ip:8000/mcp"]
    }
  }
}
// mcp-remote is a bridge that wraps HTTP servers for STUDIO-only clients
```

FastAPI → MCP Conversion

FastMCP supports converting an existing **FastAPI** application into an MCP server. This lets businesses expose their existing REST API to AI clients without rewriting logic:

```
# fastapi to mcp.py
from fastapi import FastAPI
from fastmcp import FastMCP
app = FastAPI()
@app.get('/expenses')
def get_expenses():
    @app.post('/expenses')
def create_expense(amount: float, category: str):
    # Convert FastAPI app to MCP server
    mcp = FastMCP.from_fastapi(app, name='expense-api-mcp')
if __name__ == '__main__':
    mcp.run(transport='streamable-http', port=9000)
```

■ *FastAPI→MCP conversion is a huge business advantage: existing REST backends instantly become MCP-compatible without code duplication.*

10. Building MCP Clients

While Claude Desktop is a pre-built MCP client, you can build your own custom MCP client — for example, a Python agent that programmatically talks to multiple MCP servers. This is covered in the CampusX 'How to Build MCP Clients' video.

```
# Custom MCP Client — Connecting to Multiple Servers
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    # Connect to local Math MCP server
    math_params = StdioServerParameters(
        command='npython', args=['math server.py']
    )
    async with stdio_client(math_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            # Discover available tools
            tools = await session.list_tools()
            print('Tools:', [t.name for t in tools.tools])
            # Call a tool
            result = await session.call_tool('add', {'a': 5, 'b': 3})
            print('5 + 3 =', result.content[0].text)

asyncio.run(main())
```

```
# Connecting to a Remote MCP Server (HTTP)
from mcp.client.streamable_http import streamable_http_client

async def connect_remote():
    async with streamable_http_client('http://myserver:8000/mcp') as (r, w):
        async with ClientSession(r, w) as session:
            await session.initialize()
            tools = await session.list_tools()
            # ... use tools
```

■ A single MCP client can maintain sessions with MULTIPLE MCP servers simultaneously. The LLM picks which server's tools to call based on context.

11. MCP Inspector — Debugging Tool

What is MCP Inspector?

MCP Inspector is the **official browser-based debugging tool** for MCP servers. It lets you connect to any MCP server, list its tools/resources/prompts, and invoke them manually — without needing Claude Desktop.

Running MCP Inspector

```
# Terminal — launch inspector
# Start your server first
```

```
python expense_tracker_server.py  
# In another terminal launch inspector  
npx @modelcontextprotocol/inspector  
# Open browser at http://localhost:5173  
# Connect to: http://localhost:8000/mcp (or stdio transport)
```

Inspector Capabilities

- List all tools, resources, and prompts exposed by the server.
- Call any tool with custom JSON input and see the response.
- Read any resource and view its content.
- View protocol-level JSON-RPC messages in real time.
- Test error handling and edge cases before connecting to a real LLM.

12. MCP Ecosystem & Real-World Use Cases

Official MCP Servers (maintained by Anthropic & community)

- **Filesystem** — read/write local files with permission controls.
- **GitHub** — manage repos, PRs, issues, and code via natural language.
- **Slack** — send messages, read channels, search workspace.
- **Google Drive** — search and read documents, spreadsheets.
- **PostgreSQL / SQLite** — natural language to SQL query execution.
- **Sentry** — fetch error reports and stack traces into LLM context.
- **Puppeteer** — browser automation via LLM instructions.
- **Memory** — persistent knowledge graph the LLM updates over time.

CampusX Real-World Demo — AI Newsletter Generator

In the Trilogy intro video, CampusX demonstrated an end-to-end MCP workflow: the AI was given a topic and, using MCP, it autonomously — (1) **researched** the topic via web-search tools, (2) **drafted** the newsletter copy, and (3) **designed** the layout — all without manual copy-pasting. Each phase used a different MCP server.

Benefits Summary

Benefit	Explanation
Interoperability	Any MCP-compatible host works with any MCP server. Write once, use everywhere.
Reduced Integration Cost	M+N integrations instead of M×N. Huge savings at scale.
Security	Servers handle auth; LLM never sees raw credentials.
Discoverability	LLM can discover tools at runtime — no hard-coded prompts.
Agentic Power	LLM can chain multiple tool calls across servers to complete complex tasks.
Observability	Centralised server-side logging, monitoring, and rate limiting.

13. Interview Q&A; — 30 Questions

Q: [Q1] What is the Model Context Protocol?

A: MCP is an open, vendor-neutral protocol by Anthropic (2024) that standardises how AI applications connect to external tools, data sources, and services using a client-server architecture over JSON-RPC 2.0.

Q: [Q2] Why was MCP created?

A: To solve the $M \times N$ integration problem — without MCP, each AI app needed custom code to talk to each data source. MCP reduces this to $M+N$ by providing a universal interface layer.

Q: [Q3] What analogy is used to explain MCP?

A: 'USB-C for AI' — just as USB-C standardised peripheral connections for devices, MCP standardises how LLMs connect to external systems.

Q: [Q4] What are the three roles in MCP architecture?

A: MCP Host (the AI application), MCP Client (protocol client inside the host, 1:1 with server), and MCP Server (exposes tools/resources/prompts).

Q: [Q5] What are the three MCP primitives?

A: Tools (executable functions, model-controlled), Resources (read-only data, application-controlled), and Prompts (reusable templates, user-controlled).

Q: [Q6] What is the difference between Tools and Resources?

A: Tools are action-oriented and can read AND write data; the LLM decides when to call them. Resources are strictly read-only context; the application decides when to surface them.

Q: [Q7] What is the MCP lifecycle?

A: Five phases: (1) Discovery, (2) Initialisation (version handshake), (3) Capability Exchange (list tools/resources/prompts), (4) Active Session, (5) Termination.

Q: [Q8] What are the two MCP transports and when do you use each?

A: STDIO for local servers on the same machine (Claude Desktop use case). Streamable HTTP for remote/cloud servers accessed by multiple clients.

Q: [Q9] What is FastMCP?

A: FastMCP is the official high-level Python SDK for building MCP servers. It provides Pythonic decorators (`@mcp.tool`, `@mcp.resource`, `@mcp.prompt`) built on top of the low-level MCP SDK.

Q: [Q10] How do you define a tool in FastMCP?

A: Using the `@mcp.tool()` decorator on a Python function with a docstring (description) and typed parameters (JSON schema generated automatically).

Q: [Q11] What is MCP Inspector and why is it useful?

A: MCP Inspector is a browser-based debugging tool (`npx @modelcontextprotocol/inspector`) that lets developers connect to any MCP server, discover capabilities, and manually invoke tools without needing Claude Desktop.

Q: [Q12] How does MCP differ from traditional function calling?

A: Function calling is LLM-specific and requires hard-coded schemas. MCP is vendor-neutral, has a standardised lifecycle, built-in discovery (`tools/list`), stateful sessions, and covers both action (`tools`) and context (`resources`) primitives.

Q: [Q13] What protocol underlies MCP message exchange?

A: JSON-RPC 2.0 — a lightweight remote procedure call protocol using JSON for message encoding.

Q: [Q14] What is the copy-paste hell problem MCP solves?

A: Before MCP, developers had to manually copy context (code, documents, records) into AI chat windows. MCP lets the AI autonomously pull the right context from authoritative servers, eliminating manual copy-pasting.

Q: [Q15] Can one MCP client talk to multiple servers?

A: Yes. A single MCP host can maintain MCP client sessions with multiple servers simultaneously. The LLM selects the appropriate server's tools based on the task.

Q: [Q16] What happens during Capability Exchange?

A: The client calls `tools/list`, `resources/list`, and `prompts/list` to dynamically discover everything the server exposes. This allows the LLM to know what actions are available at runtime.

Q: [Q17] How is auth handled in remote MCP servers?

A: Auth is server-side. Remote servers use OAuth 2.0, Bearer tokens, or API keys. The LLM never sees raw credentials — the server handles all auth logic.

Q: [Q18] What is the difference between SSE and Streamable HTTP in MCP?

A: SSE (deprecated, spec 2024-11-05) needed two endpoints — a GET for the event stream and a POST for client messages. Streamable HTTP (spec 2025-03-26) unified this into a single `/mcp` endpoint.

Q: [Q19] Why must STDIO servers log to stderr, not stdout?

A: MCP uses stdout for JSON-RPC messages. Any output to stdout (like `print` statements) corrupts the protocol stream and causes the client to disconnect.

Q: [Q20] How do you connect Claude Desktop to a local MCP server?

A: Add an entry in `claude_desktop_config.json` under `'mcpServers'` with the command and args pointing to your server script. Claude Desktop spawns it as a child process on startup.

Q: [Q21] How do you connect Claude Desktop to a remote MCP server?

A: Use mcp-remote as a bridge: set command to 'npx', args to ['-y', 'mcp-remote', 'http://server-url/mcp'] in claude_desktop_config.json.

Q: [Q22] What is FastAPI→MCP conversion and why is it useful?

A: FastMCP.from_fastapi(app) converts an existing FastAPI application into an MCP server. This allows businesses to expose existing REST APIs to AI clients without rewriting any business logic.

Q: [Q23] What is the MCP ecosystem?

A: A growing collection of official and community MCP servers (GitHub, Slack, Filesystem, PostgreSQL, Google Drive, etc.) and clients (Claude Desktop, Cursor, VS Code). Open-sourced under Linux Foundation.

Q: [Q24] What does 'model-controlled' mean for Tools?

A: The LLM autonomously decides WHEN to call a tool based on the conversation. The developer does not need to explicitly trigger it — the model infers the right tool from context.

Q: [Q25] What does 'application-controlled' mean for Resources?

A: The host application decides which resources to surface to the LLM, not the LLM itself. Resources are injected into context by the application layer.

Q: [Q26] What endpoint does a client use to invoke a tool?

A: tools/call — with the tool name and input parameters as JSON. The server returns the result which the LLM reads and incorporates into its response.

Q: [Q27] How does MCP enable agentic AI?

A: MCP gives AI models a standardised way to discover actions (tools), access context (resources), and follow workflows (prompts). This enables autonomous, multi-step agentic loops: reason → retrieve → act → iterate.

Q: [Q28] What is the MCP Server Inspector and what's its npm command?

A: The official browser UI for testing MCP servers: npx @modelcontextprotocol/inspector. It connects to your server and lets you call tools, read resources, and view raw protocol messages.

Q: [Q29] How are resources different from direct database queries?

A: Resources are abstracted, read-only data access with defined URIs (e.g., 'config://categories'). The LLM accesses data through a safe, server-controlled interface rather than executing raw SQL.

Q: [Q30] What is the significance of MCP being under the Linux Foundation?

A: It guarantees long-term vendor neutrality, open governance, and community ownership. No single company (including Anthropic) can unilaterally change the protocol, making it safe for enterprise adoption.

14. Quick-Reference Cheat Sheet

Key Terms at a Glance

Term	One-Line Definition
MCP	Open protocol for connecting AI apps to external tools & data (USB-C for AI)
MCP Host	The AI application (e.g., Claude Desktop) that embeds an MCP Client
MCP Client	Protocol client inside a Host; maintains 1:1 session with a Server
MCP Server	Lightweight process exposing tools/resources/prompts over MCP
Tool	Executable function the LLM calls to take action (model-controlled)
Resource	Read-only data the app surfaces to the LLM (application-controlled)
Prompt	Reusable instruction template the user selects (user-controlled)
STDIO	Local transport — client spawns server as child process
Streamable HTTP	Remote transport — HTTP + SSE for multi-client server access
FastMCP	Official high-level Python SDK with @mcp.tool / @mcp.resource decorators
JSON-RPC 2.0	The underlying wire protocol for all MCP messages
MCP Inspector	Browser-based debug tool: npx @modelcontextprotocol/inspector
Capability Exchange	Client lists tools/resources/prompts from server at session start
mcp-remote	npm bridge that lets STDIO-only clients connect to remote HTTP servers

Must-Know Commands

```
# Install FastMCP
pip install fastmcp
pip install mcp          # low-level SDK
```

```
# Run MCP Inspector
npx @modelcontextprotocol/inspector
```

```
# FastMCP minimal server (3 lines)
from fastmcp import FastMCP
mcp = FastMCP('demo')
@mcp.tool()
def hello(name: str) -> str: return f'Hello {name}'
mcp.run()
```

```
# FastMCP run modes
mcp.run()                # STDIO (local)
mcp.run(transport='streamable-http', port=8000) # Remote HTTP
```

